Давай напишем тест для проверки функций активыации (обучение с эталонным и пргнорзным Y массивами).
Вот шаблоны для обучения:

```
using System;
using System.Collections.Generic;
using System.Linq;
using NOpenCL;
using cstorchcl.core;
using cstorchcl.compiler;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("=== ТЕСТИРОВАНИЕ ОБУЧАЕМОГО СЛОЯ LINEAR (OPENCL JIT)
===");

        try
        {
            // 1. Инициализация OpenCL
            using var computeContext = new ComputeContext(preferAccelerator:
true);
            using var queue =
computeContext.Context.CreateCommandQueue(computeContext.Device);
            DeviceQueueRegistry.RegisterCurrent(queue);

            Console.WriteLine($"Выбран девайс: {computeContext.Device.Name}");

            // Геометрия: Батч = 2 примера, Входных фичей = 3, Выходных фичей =
4
            int batch = 2; int inFeatures = 3; int outFeatures = 4;

            float[] inputData = new float[] {
                1.0f, 2.0f, 3.0f,  // Пример 1
                4.0f, 5.0f, 6.0f   // Пример 2
            };
            float[] targetData = new float[] {
                0.0f, 1.0f, 0.0f, 0.0f,  // Желаемый выход для Примера 1
                1.0f, 0.0f, 0.0f, 0.0f   // Желаемый выход для Примера 2
            };

            // Инициализация тензоров данных
            using var xTensor = new Tensor(inputData, new[] { batch, 1,
inFeatures }, computeContext.Context, MemoryFlags.ReadWrite, requiresGrad:
true);
            using var targetTensor = new Tensor(targetData, new[] { batch, 1,
outFeatures }, computeContext.Context, MemoryFlags.ReadWrite, requiresGrad:
false);

            var nnClGlob = new NnClGlob(computeContext);

            // 2. Создание нашего нового слоя Linear!
            using var linearLayer = new Linear(computeContext, nnClGlob,
inFeatures, outFeatures);

            // Передаем параметры слоя в оптимизатор SGD
            var optimizer = new SGD(linearLayer.GetParameters(), learningRate:
0.2f, computeContext);

            Console.WriteLine("\n--- Выполнение шага обучения слоя Linear ---");
            for (int epoch = 0; epoch < 10; epoch++)
            {
                Console.WriteLine($"эпоха {epoch+1}!");
```

```
                GradientStorage.Clear();
                linearLayer.ZeroGrad(); // Сбрасываем старые буферы градиентов параметров

                // А. Прямой проход слоя (BMM + JIT Bias)
                Tensor pred = linearLayer.Forward(xTensor);
                Console.WriteLine($"[Forward] Выходной тензор успешно рассчитан. Форма: [{string.Join(", ", pred.Shape)}]");

                // Б. Расчет величины ошибки
                Tensor loss = nnClGlob.MseLoss(pred, targetTensor);
                float[] lossCpu = HostReturner.ToCpu(loss);
                Console.WriteLine($"[Loss] Текущая ошибка слоя: {lossCpu[0]:F6}");

                // В. ЗАПУСК ОБРАТНОГО ПРОХОДА (Autograd)
                // Градиент пойдет: MSE -> LocalOp (дельта по bias) -> BmmOp -> Веса Weight -> Входной тензор X
                AutoGrad.Backward(loss, computeContext.Context);
                Console.WriteLine("[Backward] Каскадный обратный проход графа успешно завершен!");

                // Г. Проверка сцепки буферов градиентов на GPU во VRAM
                Console.WriteLine("\nАнализ буферов градиентов после сцепки:");
                Console.WriteLine($"  -> Градиент для смещения Bias создан: {linearLayer.Bias.Grad != null}");
                Console.WriteLine($"  -> Градиент для матрицы весов Weight создан: {linearLayer.Weight.Grad != null}");
                Console.WriteLine($"  -> Градиент дотек до самого первого входа X: {xTensor.Grad != null}");

                if (linearLayer.Bias.Grad != null && linearLayer.Weight.Grad != null)
                {
                    float[] biasGrad = HostReturner.ToCpuGrad(linearLayer.Bias);
                    float[] weightGrad = HostReturner.ToCpuGrad(linearLayer.Weight);

                    Console.ForegroundColor = ConsoleColor.Green;
                    Console.WriteLine($"\n[Успех] Градиент Bias (все 4 эл.): [{string.Join(", ", biasGrad.Select(x => x.ToString("F4")))}]");
                    Console.WriteLine($"        Градиент весов Weight (первые 4): {weightGrad[0]:F5}, {weightGrad[1]:F5}, {weightGrad[2]:F5}, {weightGrad[3]:F5}");
                    Console.ResetColor();

                    // Д. Физическое обновление параметров во VRAM
                    optimizer.Step();
                    Console.WriteLine("\n[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.");
                }
                else
                {
                    Console.ForegroundColor = ConsoleColor.Red;
                    Console.WriteLine("\n[Ошибка] Градиенты застряли на стыке слоев!");
                    Console.ResetColor();
                }
}
                Console.WriteLine("\n=== ТЕСТ ОБУЧАЕМОГО СЛОЯ LINEAR С УСПЕХОМ ЗАВЕРШЕН! ===");

        }
        catch (Exception ex)
```

```
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"\n[КРИТИЧЕСКАЯ ОШИБКА]: {ex.Message}");
            Console.ResetColor();
        }
        finally
        {
            DeviceQueueRegistry.Clear();
        }

        Console.ReadLine();
    }
}
```

```
using System;
using System.Collections.Generic;
using NOpenCL;
using cstorchcl.core;
using cstorchcl.compiler;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("=== ЗАПУСК ТЕСТА BROADCASTING (JIT MAT + VECTOR)
===");

        //try
        {
            // 1. Инициализация OpenCL контекста и очереди
            using var computeContext = new ComputeContext(preferAccelerator:
true);
            using var queue =
computeContext.Context.CreateCommandQueue(computeContext.Device);
            DeviceQueueRegistry.RegisterCurrent(queue);

            Console.WriteLine($"Выбран ускоритель:
{computeContext.Device.Name}");

            // Геометрия тензоров для честной проверки Broadcasting
            int[] matrixShape = new[] { 3, 4 }; // Двумерная матрица (3 строки,
4 колонки)
            int[] vectorShape = new[] { 1, 4 }; // Одномерный вектор (1 строка,
4 колонки) — будет растягиваться по строкам!

            // Исходные данные на CPU
            float[] matrixData = new float[] {
                1.0f,  2.0f,  3.0f,  4.0f,
                5.0f,  6.0f,  7.0f,  8.0f,
                9.0f, 10.0f, 11.0f, 12.0f
            };
            float[] vectorData = new float[] { 0.5f, 1.0f, -0.5f, 2.0f }; //
bias для каждой колонки
            float[] targetData = new float[12]; // Идеальный целевой таргет
(заполнен нулями)
            Array.Fill(targetData, 0.0f);

            // Инициализация тензоров во VRAM с расчетом Strides под капотом
            using var tensorA = new Tensor(matrixData, matrixShape,
computeContext.Context, MemoryFlags.ReadWrite, requiresGrad: true);
            using var tensorBias = new Tensor(vectorData, vectorShape,
computeContext.Context, MemoryFlags.ReadWrite, requiresGrad: true);
            using var targetTensor = new Tensor(targetData, matrixShape,
computeContext.Context, MemoryFlags.ReadWrite, requiresGrad: false);

            var nnClGlob = new NnClGlob(computeContext);
            var optimizer = new SGD(new[] { tensorA, tensorBias }, learningRate:
0.01f, computeContext);

            // Собираем аргументы для JIT-конвейера
            var tensorArgs = new Dictionary<string, Tensor> {
                { "A", tensorA },
                { "bias", tensorBias }
            };

            // Наш математический псевдокод, проверяющий Broadcasting
```

```
            // Вектор bias автоматически прибавится к каждой строке матрицы A!
            string pseudoCode = @"
                A = A + bias;
                A = nnclloc.tanh(A);
            ";

            Console.WriteLine("\nЗапуск шага обучения с Broadcasting...");
            GradientStorage.Clear();

            // 1. Прямой проход (Forward JIT)
            // Размер глобальной сетки потоков автоматически выставится по
матрице A (12 потоков)
            NnClLoc.Run(pseudoCode, tensorArgs,computeContext, constants: null);

            // 2. Считаем ошибку MSE Loss
            Tensor loss = nnClGlob.MseLoss(tensorA, targetTensor);
            float[] lossCpu = HostReturner.ToCpu(loss);
            Console.WriteLine($"[Шаг 1] Текущий Loss: {lossCpu[0]:F6}");

            // Скачаем и выведем матрицу A до обновления весов, чтобы увидеть
результат сложения
            float[] matrixBefore = HostReturner.ToCpu(tensorA);
            Console.WriteLine("\nЗначения первой строки матрицы A после
Broadcasting (но до шага SGD):");
            Console.WriteLine($"  [{matrixBefore[0]:F4}, {matrixBefore[1]:F4},
{matrixBefore[2]:F4}, {matrixBefore[3]:F4}]");

            // 3. Обратный проход (Backward JIT)
            // Должен сгенерировать и выполнить ядро градиентов с обратным
Broadcasting
            AutoGrad.Backward(loss, computeContext.Context);
            Console.WriteLine("Автодифференцирование графа успешно завершено!");

            // 4. Проверка буферов градиентов во VRAM
            Console.WriteLine($"\nПроверка буферов градиентов во VRAM:");
            Console.WriteLine($"  -> Градиент для матрицы A создан:
{tensorA.Grad != null}");
            Console.WriteLine($"  -> Градиент для вектора bias создан:
{tensorBias.Grad != null}");

            if (tensorBias.Grad != null)
            {
                // Если обратный Broadcasting сработал верно, градиенты от всех
3-х строк матрицы
                // должны были корректно аккумулироваться в 4 элемента вектора
bias!
                float[] gradBias = HostReturner.ToCpuGrad(tensorBias);
                Console.WriteLine($"  -> Значения саккумулированного градиента
bias: [{string.Join(", ", gradBias.Select(x => x.ToString("F4")))}]");
            }

            // 5. Обновление весов
            optimizer.Step();
            Console.WriteLine("\nШаг SGD успешно выполнен. Байты во VRAM
обновлены.");

            Console.WriteLine("\n=== ТЕСТ BROADCASTING ПРОЙДЕН УСПЕШНО! СХЕМА
СЛИЯНИЯ РАЗМЕРНОСТЕЙ СТАБИЛЬНА ===");
        }

        //finally
        {
            DeviceQueueRegistry.Clear();
        }
```

```
            Console.ReadLine();
        }
    }
```

Поскольку broadcast на данный момент работает нгесовсем корректно, давай при тестирование использовать миатрицы не требующие broadcast.